

PLDM: Planning with Latent Dynamics Models - アーキテクチャ解説

概要

PLDM (Planning with Latent Dynamics Models) は、報酬なしのオフライン軌跡から潜在空間で環境の動力学を学習するモデルベース強化学習アプローチです。このモデルはJoint Embedding Predictive Architecture (JEPA) に基づいており、Model Predictive Control (MPC) とMPPI最適化を用いてプランニングを行います。

論文: [Learning from Reward-Free Offline Data: A Case for Planning with Latent Dynamics Models](#)

1. モデルアーキテクチャ

1.1 全体構造: HJEPA (階層的JEPA)

トップレベルのモデルは HJEPA ([pldm/models/hjepa.py:24-92](#) で定義) で、`level1` と呼ばれる単一レベルのJEPAモデルをラップしています。階層構造は複数レベルの時間的抽象化をサポートするために設計されましたが、現在の設定 (`disable_l2: true`) では `level1` のみがアクティブです。

HJEPA

- └─ `level1` (JEPA)
 - └─ `backbone` (エンコーダー)
 - └─ `backbone_ema` (EMAエンコーダー、オプション)
 - └─ `predictor` (RNNベースの動力学モデル)

1.2 JEPA (Joint-Embedding Predictive Architecture)

コアとなるモデルは JEPA ([pldm/models/jepa.py:35-245](#) で定義) で、以下で構成されます：

1. **Backbone (エンコーダー)**: 生の観測を潜在表現にエンコード
2. **Backbone EMA** (オプション): 安定性のためのエンコーダーのExponential Moving Average版
3. **Predictor**: 現在の潜在状態と行動から未来の潜在状態を予測

設定例 (seqlen90_3M.yaml より)

```
hjepa:
  level1:
    backbone:
      arch: impala                                # エンコーダーアーキテクチャ
      backbone_subclass: i                        # IMPALAバリエーション
      backbone_width_factor: 2                    # 幅の乗数
      channels: 2                                  # 入力チャンネル数
      final_ln: true                              # 最終LayerNorm
    predictor:
      predictor_arch: rnnV2                       # RNNベースの予測器
      predictor_subclass: '512-512'              # 隠れ層の次元
      rnn_layers: 1                               # RNN層の数
      residual: true                              # 残差接続
      predictor_ln: true                          # 予測器のLayerNorm
    action_dim: 2                                 # 行動空間の次元
    momentum: 0                                  # EMAモメンタム (0 = 無効)
```

2. コンポーネントの詳細

2.1 Backbone (エンコーダー)

エンコーダーは生の観測を潜在表現に変換します。

アーキテクチャ: IMPALA CNN ([pdm/models/encoders/impala.py](https://github.com/DeepMind/impala/blob/master/pdm/models/encoders/impala.py))

IMPALA エンコーダーの構成：

- 畳み込み層を持つ複数の残差ブロック
- Group Normalization
- プーリングまたはストライド畳み込みによる次元削減
- オプシヨンの最終LayerNorm

入力/出力:

- 入力: (T, B, C, H, W) ここで T=時間、B=バッチ、C=チャンネル、H/W=高さ/幅
- 出力: (T, B, D) ここで D=表現次元 (通常512または1024)

設定から:

- `channels`: 2 - 入力2チャンネル (グレースケール + 追加情報)
- `img_size`: 65 - 入力画像は65×65ピクセル
- `backbone_width_factor`: 2 - 畳み込み層の幅を2倍にする

2.2 Predictor (動力学モデル)

予測器は未来の潜在状態を予測するRNNベースのモデルです。

アーキテクチャ: `RNNPredictorV2` ([pldm/models/predictors.py:328-391](#))

コンポーネント:

- **GRU Cell**: 時間的動力学のためのコアとなる再帰ユニット
- **LayerNorm**: 安定性のためにRNN出力に適用
- **残差接続**: オプションのスキップ接続

順伝播 (1ステップ):

```
def forward(self, rnn_state, rnn_input):
    # rnn_state: (num_layers, bs, hidden_dim)
    # rnn_input: (bs, action_dim)
    next_state, next_hidden_state = self.rnn(rnn_input.unsqueeze(0), rnn_state)
    next_state = self.final_ln(next_state)
    next_hidden_state = self.final_ln(next_hidden_state)
    return next_state[0], next_hidden_state
```

複数ステップのロールアウト (`forward_multiple`):

- 初期潜在状態を受け取る: z_0 (エンコーダーから)
- 行動を使ってTステップロールアウト: $a_0, a_1, \dots, a_{T-1}$
- 予測された潜在状態を生成: $\hat{z}_1, \hat{z}_2, \dots, \hat{z}_T$

設定から:

- `predictor_arch`: `rnnV2` - GRUベースの予測器
- `rnn_layers`: 1 - 単一層GRU
- `predictor_subclass`: `'512-512'` - 隠れ層次元512
- `residual`: `true` - 残差接続を有効化

3. テンソル処理フロー

3.1 訓練時 (Forward Posterior)

訓練中、モデルは真の観測シーケンスを処理します。

入力:

- `states` : (T, B, C, H, W) - 生の観測
- `actions` : (T-1, B, A) - 状態間の行動

処理パイプライン:

1. すべての状態をエンコード → 潜在表現

```
states (T, B, 2, 65, 65)
  ↓ backbone.forward_multiple()
encodings (T, B, D)
```

- エンコーダーを通してシーケンス全体を処理
- 各タイムステップは独立にエンコード
- 出力: $z_0, z_1, \dots, z_T$

2. オプション: EMAバックボーンでエンコード (momentum > 0 の場合)

```
states (T, B, 2, 65, 65)
  ↓ backbone_ema.forward_multiple()
ema_encodings (T, B, D)
```

3. 未来の状態を予測 → 予測された潜在シーケンス

```
z_0 (B, D), actions (T-1, B, A)
  ↓ predictor.forward_multiple()
predictions (T, B, D)
```

- 初期状態 z_0 からスタート
- 行動を使って予測器をロールアウト
- 出力: $\hat{z}_0, \hat{z}_1, \dots, \hat{z}_T$ (ここで $\hat{z}_0 = z_0$)

コードの場所: [pldm/models/jepa.py:161-236](#)

3.2 プランニング時 (Forward Prior)

プランニング/評価時には、モデルは単一のエンコードされた観測からロールアウトします。

入力:

- `input_states` : (B, C, H, W) または (B, D) - 単一の観測または潜在状態
- `actions` : (T, B, A) - 計画された行動シーケンス

処理パイプライン:

1. 初期状態をエンコード (まだエンコードされていない場合)

```
input_states (B, 2, 65, 65)
  ↓ backbone.forward_multiple()
current_state (B, D)
```

2. 予測をロールアウト

```
current_state (B, D), actions (T, B, A)
  ↓ predictor.forward_multiple()
predictions (T, B, D)
```

- T個の未来の潜在状態を予測
- プランニングに使用: 行動シーケンスを評価

コードの場所: [pldm/models/jepa.py:110-159](#)

3.3 シーケンス長とサブサンプリング

設定から:

- `n_steps`: 16 - 訓練シーケンス長
- `l1_n_steps`: 16 - レベル1のシーケンス長 (`n_steps`と異なる場合もある)

訓練中、オフラインデータセットがより長いシーケンスを持つ場合、HJEPAはランダムにサブシーケンスをサンプリングします:

```
# HJEPA.forward_posterior 内 (hjepa.py:59-89)
sub_idx = random.randint(0, input_states.shape[0] - self.config.l1_n_steps)
l1_input_states = input_states[sub_idx : sub_idx + self.config.l1_n_steps]
l1_actions = actions[sub_idx : sub_idx + self.config.l1_n_steps - 1]
```

これは以下を意味します：

- 完全な軌跡は90ステップかもしれない（データセットから）
- モデルはランダムな16ステップのサブシーケンスで訓練
- 特定の軌跡セグメントへの過学習を防ぐ

4. 損失関数とハイパーパラメータ

モデルは主に2つの目的関数で訓練されます：**VICReg** と **IDM**。

4.1 VICReg (分散-不変性-共分散正則化)

VICRegは3つのコンポーネントを通じて表現の崩壊を防ぎます。

実装: [pldm/objectives/vicreg.py:54-199](#)

コンポーネント：

1. 類似度損失 (予測とターゲット間のMSE)

```
sim_loss = (ema_encodings[1:] - state_predictions[1:]).pow(2).mean()
```

- 予測された潜在状態とエンコードされた真の潜在状態間の距離を最小化
- ターゲットはEMAエンコーダー（有効な場合）または通常のエンコーダーから
- タイムステップ1からTまでのみ計算（初期状態は除く）

2. 分散損失 (特徴の変動を促進)

```
std = sqrt(x.var(dim=1) + 0.0001) # バッチ全体の標準偏差
std_loss = mean(relu(std_margin - std))
```

- 初期状態のエンコーディング z_0 に対して計算
- 定数表現への崩壊を防ぐ
- $std < std_margin$ の場合にペナルティ

3. 共分散損失 (特徴の冗長性を削減)

```
cov = einsum("bki,bkj->bij", x, x) / (batch_size - 1)
cov_loss = (cov.pow(2).sum() - diagonals) / num_features
```

- 初期状態のエンコーディング z_0 に対して計算
- 特徴次元間の非相関を促進
- 非対角要素が小さくあるべき

オプション: 時間的正則化

- `sim_loss_t`: 連続するエンコーディング間の滑らかさ
- `std_loss_t`: 時間次元にわたる分散
- `cov_loss_t`: 時間次元にわたる共分散

VICReg総損失:

```
total_loss = (
    sim_coeff * sim_loss +
    std_coeff * std_loss +
    cov_coeff * cov_loss +
    sim_coeff_t * sim_loss_t +
    std_coeff_t * std_loss_t +
    cov_coeff_t * cov_loss_t
)
```

ハイパーパラメータ (seqlen90_3M.yaml より):

```
objectives_l1:
  vicreg:
    sim_coeff: 1.0          # 予測類似度の重み
    std_coeff: 3.9843       # 分散正則化の重み
    cov_coeff: 6.9238       # 共分散正則化の重み
    std_coeff_t: 0.24535    # 時間的分散の重み
    cov_coeff_t: 0.0        # 時間的共分散の重み (無効)
    sim_coeff_t: 0.74242    # 時間的滑らかさの重み
    std_margin: 1.0         # 最小標準偏差
    std_margin_t: 1.0       # 最小時間的標準偏差
    adjust_cov: true        # 共分散を (num_features - 1) で正規化
```

4.2 IDM (逆動力学モデル)

IDMは、連続する潜在状態から行動を予測する補助ネットワークを訓練します。

実装: [pldm/objectives/idm.py:56-116](#)

目的: 潜在表現が行動に関連する情報をエンコードすることを保証。

アーキテクチャ: 連結された状態ペアを受け取るMLP

入力: $[z_t, z_{t+1}]$ (連結)
↓ MLP ($2 \times D \rightarrow$ 隠れ層 $\rightarrow$ action_dim)
出力: predicted_action

損失:

```
action_loss = MSE(predicted_actions, ground_truth_actions)
total_loss = coeff * action_loss
```

処理:

1. 連続するエンコーディングを抽出: $z_0, z_1, \dots, z_{T-1}$ と $z_1, z_2, \dots, z_T$
2. ペアを連結: $[z_t \parallel z_{t+1}]$
3. 行動を予測: $\hat{a}_t = \text{MLP}([z_t \parallel z_{t+1}])$
4. 真の行動と比較: a_t

ハイパーパラメータ (seqlen90_3M.yaml より):

```
objectives_l1:
  idm:
    coeff: 1.072          # IDM損失の全体的な重み
    action_dim: 2         # 行動空間の次元
    arch: '512'           # MLPアーキテクチャ (隠れ層次元512)
    arch_subclass: a      # アーキテクチャバリエーション
    use_pred: false       # 予測状態を使用 (false = エンコード状態を使用)
```

4.3 統合訓練損失

最終的な訓練損失は両方の目的関数を組み合わせます：


```
# train.py:375-383 より
loss_infos = [
    objective(batch, [forward_result.level1])
    for objective in self.objectives_l1
]
total_loss = sum([loss_info.total_loss for loss_info in loss_infos])
```

ここで：

- objectives_l1 = [VICReg, IDM] (設定から: objectives: [VICReg, IDM])
- それぞれが重み付き損失を返す
- 勾配はモデル全体に逆伝播

典型的な損失の大きさ (訓練から):

- VICReg sim_loss: ~0.01 - 0.1
- VICReg std_loss: ~0.0 - 0.5
- VICReg cov_loss: ~0.1 - 1.0
- IDM action_loss: ~0.001 - 0.01

5. 訓練手順

5.1 データローディング

データセット: .npz 形式で保存されたオフライン軌跡

設定 (seqlen90_3M.yaml より):

```
data:
  dataset_type: DatasetType.Wall
  offline_wall_config:
    offline_data_path: "/pldm_envs/wall/presaved_datasets/wall-visual-config_rand_exper
    n_steps: 16          # シーケンス長
    batch_size: 64       # バッチサイズ
    img_size: 65         # 画像解像度
    lazy_load: false     # すべてのデータをメモリにロード
```

バッチ形式:

- states : (B, T, C, H, W) - 画像観測
- actions : (B, T-1, A) - 行動
- オプション: 固有受容状態、速度など

5.2 訓練ループ

場所: [pldm/train.py:313-442](#)

外側のループ (エポック):

```
epochs: 2 # 設定から (通常ははるかに高い、例: 100)
```

内側のループ (バッチ):

データセット内の各バッチに対して:

1. バッチをロードしてGPUに移動

```
s = batch.states.cuda().transpose(0, 1) # (B,T,C,H,W) → (T,B,C,H,W)
a = batch.actions.cuda().transpose(0, 1) # (B,T-1,A) → (T-1,B,A)
```

2. 順伝播

```
forward_result = self.model.forward_posterior(s, a, **optional_fields)
```

- すべての状態をエンコード → encodings
- 潜在シーケンスを予測 → predictions
- 両方を含む ForwardResult を返す

3. 損失を計算

```
loss_infos = [
    vicreg_objective(batch, [forward_result.level1]),
    idm_objective(batch, [forward_result.level1])
]
total_loss = sum([info.total_loss for info in loss_infos])
```

4. 逆伝播と最適化

```
self.optimizer.zero_grad()
total_loss.backward()
self.optimizer.step()
self.model.update_ema() # EMAが有効な場合、EMAエンコーダーを更新
```

5. ログ記録 (100ステップごと)

- 損失値
- 学習率
- 訓練/データローディング時間

環境とのインタラクションなし: モデルは訓練中に環境と一切インタラクトせず、オフラインデータセットからのみ学習します。

5.3 訓練ハイパーパラメータ

seqlen90_3M.yaml より:

```
# 最適化
base_lr: 0.0007                # 学習率
optimizer_type: Adam           # オプティマイザー
optimizer_schedule: Cosine     # 学習率スケジュール (デフォルト、この設定にはない)
epochs: 2                      # 訓練エポック (テスト設定、通常は100)

# バッチ設定
data.offline_wall_config.batch_size: 64
n_steps: 16                    # バッチあたりのシーケンス長

# チェックポイントと評価
save_every_n_epochs: 5        # チェックポイント頻度 (デフォルト)
eval_every_n_epochs: 20       # 評価頻度 (デフォルト)
eval_during_training: false    # 訓練中は評価しない
```

エポックあたりの訓練ステップ:

- データセットサイズ: 20,000軌跡 (data.wall_config.size: 20000 より)
- バッチサイズ: 64
- エポックあたりのステップ: $20,000 / 64 \approx 312$ ステップ

総訓練量:

- epochs: 100 の場合、総ステップ $\approx 31,200$
- 各ステップは $64 \text{ 軌跡} \times 16 \text{ タイムステップ} = 1,024 \text{ 状態-行動ペア}$ を処理

5.4 モデル更新

パラメータ更新:

- backbone と predictor のパラメータに対する標準的なSGD/Adamステップ
- VICRegとIDM両方の損失から勾配が流れる

EMA更新 (momentum > 0 の場合):

```
# JEPa.update_ema() 内 (jepa.py:238-245)
for param, ema_param in zip(backbone.parameters(), backbone_ema.parameters()):
    ema_param.data = momentum * ema_param.data + (1 - momentum) * param.data
```

- 現在の設定では: momentum: 0 → EMA無効

5.5 オンラインデータ収集なし

PLDMの重要な特徴:

- 訓練中の環境ステップなし
- 報酬信号不使用
- 事前収集された軌跡からの純粋なオフライン学習

データセットは一度収集され（エキスパートまたはランダムポリシーを介して）保存されます。訓練はこの静的データセットから読み取るのみです。

6. プランニング（推論）

6.1 プランニングアルゴリズム

テスト時、PLDMは **MPPI** (Model Predictive Path Integral) 最適化を用いた **Model Predictive Control (MPC)** を使用します。

設定 (seqlen90_3M.yaml より):

```

eval_cfg:
  wall_planning:
    level1:
      planner_type: PlannerType.MPPI
      max_plan_length: 96          # プランニング horizon
      mppi:
        noise_sigma: 12           # 行動ノイズの標準偏差
        num_samples: 2000         # 候補軌跡の数
        lambda_: 0.005            # 温度パラメータ
        z_reg_coeff: 0            # 潜在正則化（無効）

```

6.2 プランニングプロセス

各環境ステップで：

1. 現在の観測をエンコード

$\text{obs} (1, C, H, W) \rightarrow \text{backbone} \rightarrow \mathbf{z}_t (1, D)$

2. 行動シーケンスをサンプリング

- $\text{num_samples}=2000$ 個の候補行動シーケンスを生成
- 各シーケンスは長さ $\text{max_plan_length}=96$
- 行動はガウスノイズ + オプションの平均軌跡からサンプリング

3. 各候補に対してモデルをロールアウト

各行動シーケンス A_i に対して：

$\mathbf{z}_t \rightarrow \text{predictor}(\mathbf{z}_t, A_i) \rightarrow \text{予測軌跡 } [\hat{\mathbf{z}}_{t+1}, \dots, \hat{\mathbf{z}}_{t+H}]$

4. 各軌跡のコストを評価

$\text{cost} = \text{goal_cost} + \text{uncertainty_cost}$

- **Goal cost:** 潜在空間でのターゲットまでの距離
- **Uncertainty cost:** アンサンブル予測間の分散 ($K > 1$ の場合)

5. コストによって軌跡に重みを付ける

$\text{weights} = \exp(-\text{cost} / \text{lambda}_\text{reg})$

6. 最適な行動を計算

$\mathbf{a}_t = \text{weighted_average}(\text{candidate_actions}, \text{weights})$

7. 最初の行動を実行して再プラン

- 環境で `a_t` を実行
- 新しい状態を観測
- ステップ1から繰り返し

リプランニング: デフォルトでは、毎ステップ再プラン。効率のために頻度を減らすことも可能。

7. 主要な設計選択

7.1 なぜJEPA（再構成ではなく）？

- **潜在予測**は制御に関連する特徴に焦点を当てる
- **再構成**はピクセルレベルの詳細に容量を浪費
- 経験的結果により、JEPAは再構成ベースのモデル（例：DreamerV3）を上回る

7.2 なぜVICReg？

- **表現の崩壊**を防ぐ（すべての潜在状態が同一になる）
- 対比的ペアが不要（SimCLRと異なる）
- 3つの目的のバランス：類似度、分散、共分散

7.3 なぜIDM？

- 潜在表現が**行動に関連する情報**をエンコードすることを保証
- 自己教師あり学習によってVICRegを補完
- 自己教師ありRLにおける一般的な補助タスク

7.4 なぜGRU予測器？

- 再帰構造が自然に時間的動力学を捉える
- 短いhorizonに対してTransformerよりパラメータ効率的
- LayerNorm + 残差接続が安定性を向上

7.5 報酬なし学習

- **ラベルなし**実演からの学習を可能にする
- より柔軟：同じモデルを複数のタスクに使用可能

- プランニングコスト関数はテスト時に定義（訓練時ではない）

8. まとめ表

コンポーネント	アーキテクチャ	入力形状	出力形状	パラメータ数 (概算)
Backbone	IMPALA CNN	(T, B, 2, 65, 65)	(T, B, D=512)	~1M
Predictor	GRU + LN	(B, D=512), (T-1, B, A=2)	(T, B, D=512)	~1M
IDM MLP	MLP	(B, 2*D=1024)	(B, A=2)	~0.5M
合計	-	-	-	~2.5M

損失	コンポーネント	ハイパーパラメータ	目的
VICReg	sim, std, cov	sim=1.0, std=3.98, cov=6.92	予測 + 崩壊防止
IDM	行動MSE	coeff=1.072	行動関連表現

訓練	値	説明
データセット	20k軌跡	オフライン、報酬なし
バッチサイズ	64	バッチあたりの軌跡数
シーケンス長	16	軌跡あたりのタイムステップ
エポック	2（テスト） / 100（実際）	データセット全体のパス
学習率	0.0007	Adamオプティマイザー
エポックあたりのステップ	~312	20000 / 64

プランニング	値	説明
アルゴリズム	MPPI	サンプリングベースMPC

プランニング	値	説明
サンプル数	2000	候補軌跡
Horizon	96	先読みステップ
リプランニング	毎ステップ	各行動後に再プラン

参考文献

- 論文: [Learning from Reward-Free Offline Data: A Case for Planning with Latent Dynamics Models](#)
- コード: github.com/vladisai/PLDM
- 設定: [seqlen90_3M.yaml](#)